

kit-fundacao-aidd, fable-method e impeccable sob Análise

São submodules de verdade? Servem pra este projeto? Há prova de uso real?

Agosto de 2026

Sumário

1. Achado que muda a análise: você já usa 2 destes num projeto irmão	1
2. kit-fundacao-aidd — seu, original, altíssima aderência	1
Aderência a este projeto: altíssima, e não por coincidência	2
3. fable-method — legítimo e rigoroso, mas com um buraco real	2
O buraco: seu CLAUDE.md global referencia isso, mas não está instalado	2
4. impeccable — legítimo, maduro, e resolve exatamente o “feio” desta sessão	3
5. Veredito consolidado	3
6. Fontes	3

1. Achado que muda a análise: você já usa 2 destes num projeto irmão

Antes de avaliar cada um isoladamente: proj_fabrica-de-livros (seu, Python, push mais recente que este projeto) **já tem .gitmodules configurado** com:

```
.claude/mcp-servers/code-review-graph → tirth8205/code-review-graph.git
.claude/skills/impeccable              → pbakaus/impeccable.git (upstream, não seu
fork)
tooling/kit-fundacao-aidd              → Heverton-web/kit-fundacao-aidd.git (seu)
```

Ou seja: **2 dos 3 repos que você pediu pra analisar já estão em produção real** noutro projeto seu com arquitetura parecida com esta (outra “Fábrica”). Isso não é mais teoria — é prova de uso.

2. kit-fundacao-aidd — seu, original, altíssima aderência

Não é fork — é seu, privado, Python. README descreve 5 decisões de engenharia já maduras, extraídas de um projeto real de produção agêntica (a “Fábrica Agêntica de Publicações” — possivelmente ancestral direto deste projeto ou do proj_fabrica-de-livros) e generalizadas para instalar em qualquer projeto:

1. **Builder ≠ Critic** — quem gera nunca é quem aprova.
2. **Crítico determinístico** — checagem de formato/presença/contagem é script, nunca LLM avaliando a si mesmo.
3. **Registro declarativo** — tipo novo = 1 entrada num dicionário, nunca `if tipo == ...` espalhado.
4. **Nunca commitar vermelho** — hook mecânico, não promessa em texto.
5. **Postmortem que vira teste** — toda “Prevenção” nasce com teste de regressão.

O instalador (`instalar.py`) é **dry-run por padrão**, idempotente, nunca sobrescreve nada sem mostrar diff antes — desenhado com o mesmo cuidado que se espera de uma ferramenta que mexe em repositório de terceiros.

Aderência a este projeto: altíssima, e não por coincidência

Este projeto já pratica 4 das 5 peças, só que sem nomeá-las formalmente:

- Builder ≠ Critic → é exatamente `redator-*` (builder) vs. `revisor-marca` (critic) — já existe.
- Crítico determinístico → é exatamente `scripts/validar-*.py --estrito` como árbitro, já é princípio central (`AGENTS.md`).
- Nunca commitar vermelho → não há hook formal hoje, mas a suíte de testes do painel (79 testes) já cumpre esse papel manualmente a cada commit.
- Postmortem que vira teste → foi exatamente o que aconteceu nesta sessão com o bug do subprocess pendurado (`test_run_job_does_not_hang_waiting_for_orphaned_grandchild`) — descoberto, virou regressão testada, sem nomear “postmortem”.
- Registro declarativo → **esta é a única que talvez valha auditar** — os adaptadores de harness (`painel/harness_adapters/_REGISTRY`) já seguem esse padrão; vale checar se existe `if tipo == ...` espalhado em outro lugar do código da Fábrica.

Recomendo instalar como submodule aqui. O ganho não é “ganhar prática nova” — é **formalizar em nome + tooling instalável** algo que este projeto já faz bem informalmente, e ganhar o hook de “nunca commitar vermelho” que hoje não existe formalmente.

3. fable-method — legítimo e rigoroso, mas com um buraco real

Fork de Sahir619/fable-method, MIT, plugin Claude Code v1.4.0. **Prova séria:** 15 rodadas de avaliação, 260+ execuções de agente, juízes LLM cegos que verificam por diff/execução, não por ler relatório — inclusive relata os casos em que o método **não** ajudou (honestidade rara em README de skill). Quatro peças: `fable-method` (pensar), `fable-loop` (agir), `fable-judge` (provar), `fable-domain` (crescer).

O buraco: seu CLAUDE.md global referencia isso, mas não está instalado

Seu `~/claude/CLAUDE.md` (pessoal, vale pra todo projeto) diz explicitamente “aplicar o fable-method loop” e “rodar fable-judge”, apontando para `~/claude/skills/fable-method/SKILL.md`. **Esse caminho não existe na sua máquina** — não há instalação global de `fable-method/fable-judge` em lugar nenhum fora de submodule de projeto. A única cópia que existe é dentro do `.token-economy` deste projeto (`fable-method/fable-judge` são 2 das 9 skills de lá) — e não tenho como confirmar se é a mesma versão testada nas 260 execuções do fork, ou uma cópia mais antiga/divergente.

Efeito prático: em qualquer projeto seu que não tenha o `.token-economy` (ou outro submodule com `fable-method` embutido), a instrução do seu `CLAUDE.md` global de “aplicar fable-method”/“rodar fable-judge” é uma referência morta — não quebra nada visivelmente, só silenciosamente não executa o que você pediu pra sempre acontecer.

Recomendação: instalar `Heverton-web/fable-method` de verdade em `~/claude/skills/fable-method` (nível global, não por projeto) resolve o buraco pra todos os seus projetos de uma vez, não

só este. Dentro *deste* projeto especificamente, isso seria redundante com o que o `.token-economy` já entrega (a não ser que você queira garantir que é exatamente a versão testada do fork).

4. impeccable — legítimo, maduro, e resolve exatamente o “feio” desta sessão

Fork de `pbakaus/impeccable` (Paul Bakaus). Ferramenta de design para agentes de IA: 1 skill, 23 comandos (`/impeccable polish, audit, critique, bolder, quieter...`), 59 regras de detecção determinísticas (sem LLM) contra o “jeito AI-slop” — Inter em tudo, gradiente roxo-azul, card dentro de card. Já nativamente multi-harness (pastas `.claude`, `.codex`, `.cursor`, `.gemini`, `.grok`, `.opencode`, `.pi`, `.qoder`, `.trae`, `.vibe` já no próprio repo).

Aderência direta e imediata: é literalmente a ferramenta certa pro “visual feio” do painel que você reclamou nesta mesma sessão (a justificativa pro redesign de colapso + hierarquia visual). `proj_fabrica-de-livros` já usa isso — e usa o **upstream direto** (`pbakaus/impeccable`), não seu fork, o que sugere que pra esse repo específico não há necessidade de fork próprio (nenhuma customização feita).

Recomendação: instalar aqui, upstream direto (seguindo o padrão já validado no `proj_fabrica-de-livros`), e usar `/impeccable audit` no painel antes de aplicar o redesign combinado — vai apontar exatamente os “tells” genéricos que causam a sensação de “feio”, com regra determinística, não opinião.

5. Veredito consolidado

Repo	Instalar aqui?	Nível
<code>kit-fundacao-aidd</code>	Sim	Submodule deste projeto — já validado no <code>proj_fabrica-de-livros</code>
<code>impeccable</code>	Sim	Submodule deste projeto, upstream direto — já validado no <code>proj_fabrica-de-livros</code>
<code>fable-method</code>	Instalar globalmente (<code>~/ .claude/skills/</code>), não neste projeto especificamente	Conserta o buraco pra todos os seus projetos de uma vez

Diferente da análise anterior (LoopX, Agent Skills genérico, DDD/TypeScript), estes 3 não são “talvez sirvam” — **2 já estão provados em produção num projeto irmão seu**, e o terceiro conserta uma referência quebrada no seu próprio `CLAUDE.md` global. É a recomendação mais direta desta série de análises.

6. Fontes

- `Heverton-web/kit-fundacao-aidd` (privado, original)

- Heverton-web/fable-method (fork de Sahir619/fable-method)
- Heverton-web/impeccable (fork de pbakaus/impeccable)
- Heverton-web/proj_fabrica-de-livros — `.gitmodules` inspecionado diretamente, prova de uso real